

Практика по алгоритмам

Евгений Кравченко, Виктор Крыштапович, Александр Мишунин *

Весна, 2026

Содержание

1	Кратчайшие пути	3
1.1	Практика	3
2	Интервальные запросы, LCA	4
2.1	Практика	4
2.2	Домашнее задание	5
3	AVL-деревья	6
3.1	Практика	6
3.2	Домашнее задание	7
4	Деревья	8
4.1	Практика	8
4.2	Домашнее задание	10
5	Деревья и отрезки	11
5.1	Практика	11
5.2	Домашнее задание	12
6	Хеширование строк	13
6.1	Практика	13
6.2	Домашнее задание	15
7	Универсальные семейства	16
7.1	Практика	16
7.2	Домашнее задание	17
8	Алгебра	18
8.1	Практика	18
8.2	Домашнее задание	19
9	FFT	21
9.1	Практика	21
9.2	Домашнее задание	22
10	Потоки и разрезы	23
10.1	Практика	23
10.2	Домашнее задание	24
11	Линейное программирование	26
11.1	Практика	26
11.2	Домашнее задание	28

*Составители сборника не всегда являются авторами задач. Авторы не указаны в учебных целях.

12 Поиск подстроки в строке	30
12.1 Практика	30
12.2 Домашнее задание	31
13 Суффиксные структуры	32
13.1 Практика	32
13.2 Домашнее задание	33

1 Кратчайшие пути

1.1 Практика

1. Пусть даны n валют и m обменников, и i -й обменник предлагает менять валюту a_i на валюту b_i по курсу c_i/d_i . Приведите алгоритм, определяющий за время $\mathcal{O}(nm)$, возможно ли бесконечно обогащаться. Другими словами, существуют ли такие $p, q, s (p < q)$, что, имея p единиц валюты типа s , можно за несколько обменных операций получить хотя бы q единиц валюты типа s ? Считайте, что у обменников есть бесконечное количество денег целевой валюты.
2. Пусть дан взвешенный граф G (возможно, с циклами отрицательного веса). На вершинах этого графа определим функцию $\phi(v)$ — потенциал. Заменим вес каждого ребра $w(u, v)$ на $w'(u, v) = w(u, v) + \phi(u) - \phi(v)$. Докажите, что кратчайшим путям между двумя вершинами в графе с весами w' будут однозначно соответствовать кратчайшие пути в графе с весами w .
3. Пусть во взвешенном графе G нет циклов отрицательной стоимости. Докажите, что если в качестве потенциала взять кратчайшее расстояние от некоторой вершины s , то все веса w' получатся неотрицательными (если соответствующие расстояния конечны).
4. Дана система из m неравенств на n переменных x_i . Каждое неравенство имеет вид $x_j - x_i \leq \delta_{ij}$.
 - (a) Найти решение системы или сказать, что его не существует, за $\mathcal{O}(n \cdot m)$.
 - (b) Пусть все $\delta_{ij} \geq 0$, решить задачу за $\mathcal{O}(n \cdot m)$.
5. Дан произвольный взвешенный орграф, найдите расстояние от a до b (число или $\pm\infty$). $\mathcal{O}(VE)$.
6. Предподсчёт за $\mathcal{O}(V^3)$ и запрос $\langle a, b, e \rangle$ за $\mathcal{O}(1)$ — существует ли кратчайший путь из a в b , проходящий через ребро e ?
7. Для каждой вершины графа узнать, есть ли отрицательный цикл через эту вершину. $\mathcal{O}(V^3)$.
8. Для каждой пары вершин в графе найти $w[a, b]$ — такой минимальный вес, что из a в b есть путь по рёбрам, вес которых не больше $w[a, b]$. $\mathcal{O}(V^3)$.
9. Нужно научиться на запрос “уменьшился вес ребра” за $\mathcal{O}(V^2)$ пересчитывать матрицу расстояний. Считайте, что в графе не было и не появилось отрицательных циклов.

Дополнительные задачи

10. Рассмотрим граф, на каждом ребре которого написан нолик или единица. Каждому пути в этом графе соответствует некоторая бинарная строка. Строки сравниваются как двоичные числа (старшие разряды слева), среди одинаковых чисел меньше то, у которого меньше ведущих нулей. Для задачи нахождения кратчайших путей относительно такой метрики докажите корректность или приведите контрпример для алгоритма
 - (a) Флойда
 - (b) Дейкстры
 - (c) Беллмана-Форда

2 Интервальные запросы, LCA

2.1 Практика

1. Придумайте online static RMQ за
 - (a) $\mathcal{O}(n)$ на предподсчет, $\mathcal{O}(\log n)$ на запрос.
 - (b) $\mathcal{O}(n \log n)$ на предподсчет, $\mathcal{O}(1)$ на запрос.
 - (c) $\langle \mathcal{O}(n \log \log n), \mathcal{O}(1) \rangle$.
 - (d) $\langle \mathcal{O}(n \log^* n), \mathcal{O}(\log^* n) \rangle$.
 - (e) **(*)** $\langle \mathcal{O}(n\alpha(n)), \mathcal{O}(\alpha(n)) \rangle$.
2. Вам дано дерево из одной вершины. Придумайте, как online отвечать на следующие запросы:
 - Подвесить новую вершину к дереву.
 - LCA двух вершин.
 - (a) $\mathcal{O}(1)$ на подвешивание, $\mathcal{O}(h)$ на LCA (h — текущая высота).
 - (b) $\langle \mathcal{O}(\log n), \mathcal{O}(\log n) \rangle$ (n — число добавлений; считайте, что n известно заранее).
 - (c) $\langle \mathcal{O}(1), \mathcal{O}(\log n) \rangle$ (используйте скопленный список).
3. Дано подвешенное дерево из n вершин с весами на ребрах. Online-запросы:
 - (a) сумма весов на пути из a в b . $\langle \mathcal{O}(n), \mathcal{O}(1) \rangle$.
 - (b) минимальный вес на пути из a в b . $\langle \mathcal{O}(n \log n), \mathcal{O}(\log n) \rangle$.
4. В последовательности длины n из различных целых чисел с помощью дерева отрезков найдите наибольшую возрастающую подпоследовательность за $\mathcal{O}(n \log n)$.
5. Дан массив из нулей и единиц. Запросы: “поменять элемент”, “найти ближайшую слева/справа единицу к позиции i ”. $\langle \mathcal{O}(n), \mathcal{O}(\log n) \rangle$, online.
6. Дана таблица $n \times n$ с целыми числами. Поступают запросы:
 - Изменить значение в ячейке (i, j) ,
 - Посчитать сумму чисел в прямоугольнике $((l, t), (r, b))$. $\langle \mathcal{O}(n^2 \log n), \mathcal{O}(\log^2 n) \rangle$, online.

Дополнительные задачи

7. Вывести все числа $\geq X$ на отрезке $[L, R]$ массива. Static online
 - (a) $\langle \mathcal{O}(n \log n), \mathcal{O}(\log^2 n + k) \rangle$ (k — размер ответа).
 - (b) $\langle \mathcal{O}(n \log n), \mathcal{O}(\log n + k) \rangle$
8. Дан массив из n элементов, можно сделать предобработку за $\mathcal{O}(n \log n)$. Запросы: количество различных чисел на отрезке $[L, R]$. Тут будет подсказка про $prev[i]$.
 - (a) online за $\mathcal{O}(\log^2 n)$.
 - (b) offline за $\mathcal{O}(\log n)$.
 - (c) online за $\mathcal{O}(\log n)$.
9. Запросы: k -е по порядку среди различных чисел на отрезке $[L, R]$.
 - (a) offline за $\mathcal{O}(\log^3 n)$.
 - (b) online за $\mathcal{O}(\log^3 n)$.
 - (c) online за $\mathcal{O}(\log^2 n)$.

2.2 Домашнее задание

1. k -инверсией в перестановке p называется набор индексов $i_1 < i_2 < \dots < i_k$, такой, что $p[i_1] > p[i_2] > \dots > p[i_k]$. Найти число k -инверсий за $\mathcal{O}(nk \log n)$.
2. Дано дерево из одной вершины. Требуется уметь отвечать online за $\mathcal{O}(\log n)$ на запросы:
 - подвесить новую вершину u к вершине дерева v ,
 - вернуть диаметр дерева.

Диаметр дерева — длина самого длинного простого пути в дереве.

3. Дан ориентированный граф, в котором исходящая степень каждой вершины равна единице. Запросы online: из вершины v сделать k шагов вперед и вернуть куда пришли.
 - (a) Предподсчет: $\mathcal{O}(n \log k_{\max})$, время на запрос: $\mathcal{O}(\log k)$.
 - (b) Предподсчет: $\mathcal{O}(n \log n)$, время на запрос: $\mathcal{O}(\log \min(k, n))$.
4. Дана скобочная последовательность из круглых скобок длины n . Запросы:
 - является ли отрезок $[L, R]$ правильной скобочной последовательностью,
 - изменить i -ю скобку.

$\mathcal{O}(n)$ на предподсчет, $\mathcal{O}(\log n)$ на запрос, online.

5. Попробуем модифицировать идею **SparseTable** так, чтобы она работала для произвольных ассоциативных функций: предложите способ выделить $\mathcal{O}(n \log n)$ отрезков в массиве размера n так, что любой отрезок $[L, R]$ можно было представить в виде объединения $\mathcal{O}(1)$ непересекающихся выделенных отрезков. Заметим, что дерево отрезков выделяет $\mathcal{O}(n)$ отрезков, и любой отрезок представляется как объединение $\mathcal{O}(\log n)$ из них.

Дополнительные задачи

6. Придумайте, как в задаче 5 домашнего задания обойтись $\mathcal{O}(n \log \log n)$ отрезков.

3 AVL-деревья

3.1 Практика

1. Пусть вам даны два AVL-деревья T_1 и T_2 . Придумайте, как построить AVL-дерево T , являющееся объединением деревьев T_1 и T_2 за время:
 - (a) $\mathcal{O}(\text{height}(T_1) \times \text{size}(T_2))$, если $\text{size}(T_1) \geq \text{size}(T_2)$.
 - (b) $\mathcal{O}(\text{size}(T_1) + \text{size}(T_2))$
2. Дано двоичное дерево, в котором AVL-свойство выполнено везде, кроме корня.
 - (a) Пусть `root.l.h = root.r.h + 3`. Как восстановить AVL-свойство всюду за $\mathcal{O}(1)$?
 - (b) Пусть `root.l.h = root.r.h + k`. Как восстановить AVL-свойство всюду за $\mathcal{O}(k)$?
3. Придумайте `split` для AVL-деревьев за $\mathcal{O}(\log^2 n)$.
4. Придумайте `merge` для AVL-деревьев за $\mathcal{O}(\log n)$.
5. Докажите, что придуманный `split` на самом деле работает за $\mathcal{O}(\log n)$.
6. Придумайте структуру данных, позволяющую online за $\mathcal{O}(\log n)$ отвечать на следующие запросы:
 - добавить элемент x
 - удалить элемент x
 - найти i -ый по порядку элемент
 - найти порядок элемента x
7. Запросы online за $\mathcal{O}(\log n)$:
 - добавить пару $\langle x, y \rangle$
 - удалить пару $\langle x, y \rangle$
 - посчитать сумму y по всем парам таким, что $l \leq x \leq r$
8. Запросы online за $\mathcal{O}(\log n)$:
 - добавить пару $\langle x, y \rangle$
 - посчитать сумму y по всем парам таким, что $l \leq x \leq r$
 - посчитать сумму x по всем парам таким, что $l \leq y \leq r$
9. Запросы online за $\mathcal{O}(\log n)$:
 - `add(i, x)` — вставить x на позицию i , все элементы после него сдвигаются на 1 вправо
 - `del(i)` — удалить элемент на позиции i , все элементы после него сдвигаются на 1 влево
 - `sum(l, r)` — сумма всех x , для которых $l \leq i \leq r$
 - `add(l, r, value)` — добавить $value$ ко всем x , для которых $l \leq i \leq r$

Дополнительные задачи

10. Уменьшите количество дополнительной информации в AVL-дереве до двух бит на вершину.
11. Покажите, что любые два корректные дерева поиска, построенные на одном и том же множестве ключей, можно получить друг из друга последовательностью поворотов.

3.2 Домашнее задание

1. Покажите, что удаление из AVL дерева требует $\Omega(\log n)$ вращений в худшем случае.
2. Пусть в обычное несбалансированное бинарное дерево поиска добавляются **различные** элементы в порядке $x_0, x_1, \dots, x_{n-1}$. При этом на некоторые запросы может уйти $\Omega(n)$ времени. Сделайте все добавления за гарантированный $\mathcal{O}(\log n)$ на запрос.

(Подсказка: вам может понадобиться дополнительная структура данных)

3. Придумайте, как реализовать структуру данных, поддерживающую следующие операции на последовательности из чисел:

- `insert(i, x)` — добавление числа x на позицию i (если позиция была занята, то число, которое на ней было, и все числа правее сдвигаются вправо на 1),
- `twiddle(l, r)` — обмен местами последовательных пар соседних чисел на отрезке $[l, r]$ **четной** длины (пример: $[1, 2, 3, 4, 5, 6], l = 2, r = 5 \rightarrow [1, 3, 2, 5, 4, 6]$),
- `get(i)` — вывод числа на позиции i .

$\mathcal{O}(\log n)$ на запрос, online (n — текущая длина последовательности).

4. Придумайте структуру данных на основе AVL дерева, реализующую список с online операциями:

- `make(x0, x1, x2, ..., xn-1)` — создать список со значениями. $\mathcal{O}(n)$.
- `get(list, i)` — получить i -тый элемент списка длины n . $\mathcal{O}(\log n)$.
- `reverse(list)` — развернуть список. $\mathcal{O}(1)$.
- `splitAt(list, k)` — разделить список длины n на префикс длины k и оставшийся суффикс. $\mathcal{O}(\log n)$.
- `concat(list1, list2)` — сконкатенировать два списка суммарной длины n . $\mathcal{O}(\log n)$.

5. Дана последовательность целых чисел $a_0, a_1, \dots, a_{n-1}$. Запросы:

- `insert(i, x)` — добавление числа x на позицию i (если позиция была занята, то число, которое на ней было, и все числа правее сдвигаются вправо на 1),
- `assign(i, x)` — присвоить $a_i \leftarrow x$.
- `get(L, R)` — найти $\max_{L \leq l \leq r \leq R} \sum_{i=l}^r a_i$ (т.е. максимальную сумму подотрезка $[L, R]$).

$\mathcal{O}(\log n)$ на запрос, online (n — текущая длина последовательности).

Дополнительные задачи

6. Напишите на каком-нибудь строго типизированном функциональном языке с зависимыми типами (например, на **Agda**) реализацию исправления произвольного дисбаланса и добавления в AVL дерево так, чтобы типизацией гарантировался:

- (a) AVL-инвариант.
- (b) AVL-инвариант и порядок на ключах.

7. Постройте тест, на котором «недо-AVL-дерево», которое делает только малые вращения, делает $\omega(n \log n)$ операций после n запросов.

Формально: изначально дерево пусто, нужно n раз вызвать `add(root, xi)` для некоторой последовательности x_i , что суммарное время работы $\omega(n \log n)$.

Либо докажите, что такого теста нет.

4 Деревья

4.1 Практика

- Запросы online за $\mathcal{O}(\log n)$:
 - `add(i, x)` — вставить x на позицию i , все элементы после него сдвигаются на 1 вправо
 - `del(i)` — удалить элемент на позиции i , все элементы после него сдвигаются на 1 влево
 - `sum(l, r)` — сумма всех x , для которых $l \leq i \leq r$
 - `add(l, r, value)` — добавить $value$ ко всем x , для которых $l \leq i \leq r$
- Пусть есть произвольное `splay`-дерево, построенное на ключах $1, 2, 3 \dots n$. Покажите, что в результате последовательных вызовов `splay(1), splay(2), \dots splay(n)` получится бамбук.
- Нарисуйте все Декартовы деревья, которые могут получиться в результате операции `merge` (бамбук идущий влево-вниз, вершина) и `merge` (бамбук идущий вправо-вниз, вершина).
- Придумайте структуру данных, реализующую список с online операциями:
 - `make(x0, x1, x2, \dots xn-1)` — создать список со значениями. $\mathcal{O}(n)$.
 - `get(list, i)` — получить i -тый элемент списка длины n . $\mathcal{O}(\log n)$.
 - `concat(list1, list2)` — построить список-конкатенацию двух параметров суммарной длины n . $\mathcal{O}(\log n)$.
 - `splitAt(list, k)` — построить по списку длины n два новых списка: префикс длины k и оставшийся суффикс. $\mathcal{O}(\log n)$.
 - `copy(list, i1, i2, l)` — скопировать в списке l элементов с позиции i_1 на позицию i_2 и вернуть полученный список. $\mathcal{O}(\log n)$.
- Пусть приоритеты случайны. Покажите, что наличие одинаковых ключей не портит оценку на матожидание высоты Декартова дерева для последовательности стандартных операций `insert`.
 - Попробуем улучшить константу у стандартного `insert` следующим образом: сначала будем спускаться по дереву, пока не встретим узел с меньшим, чем у нового, приоритетом, и уже этому поддереву сделаем `split` и подвесим две части к новому узлу. Покажите, что результате всегда получится корректное Декартово дерево.
 - Придумайте аналогичный `delete` через спуск и один `merge`.
 - Пусть теперь приоритеты случайны, а ключи могут быть одинаковыми. Заметим, что при спуске у нас есть выбор, в какое поддерево идти при равенстве ключей. Влияет ли этот выбор на оценку матожидания высоты дерева?
- Покажите, что любые два корректные дерева поиска, построенные на одном и том же множестве ключей, можно получить друг из друга последовательностью поворотов.
- Придумайте модификацию `splay`-дерева, которая умеет делать `splay(key)` за один проход используя $\mathcal{O}(1)$ дополнительной памяти и при этом не хранит ссылки на предков.
- Пусть в начале есть много `splay`-деревьев суммарного размера n , над которыми затем алгоритм делает m операций `split` и `merge`. Правда ли, что суммарное время работы операций с деревьями можно оценить как $\mathcal{O}((m + n) \log n)$, если:
 - деревья персистентные.
 - деревья эфемерные.

Дополнительные задачи

9. Попробуем построить сливаемую кучу по минимуму на основе дерева следующим образом: пусть каждая вершина дерева хранит один ключ и односвязный список ссылок на дочерние поддеревья. На ключах выполняется свойство кучи. Операции будем выполнять следующим образом:

- *create-heap* — создать пустое дерево
- *find-min* — вернуть ключ, хранящийся в корне дерева
- *merge* — добавить ссылку на дерево с большим (по ключу) корнем в начало списка детей корня дерева с меньшим (если оба дерева не пустые)
- *insert* — *merge* с деревом из одного элемента
- *delete-min* — удалить корень и слить его детей таким образом: идем по списку слева направо, сливая детей попарно (1-2, 3-4, 5-6 и т. д.). Потом идем справа налево и сливаем все деревья в одно. Например, из списка детей $[c_1, c_2, c_3, c_4, c_5]$ получится дерево:

$$\text{merge}(\text{merge}(c_1, c_2), \text{merge}(\text{merge}(c_3, c_4), c_5))$$

Докажите, что любая корректная последовательность из n таких операций работает за $\mathcal{O}(n \log n)$ (в начальный момент куч нет). Подсказка: это должно быть как-то связано со *splay*-деревьями.

4.2 Домашнее задание

1. У нас есть Декартово дерево из n вершин, приоритеты случайные, а ключи все разные. Обозначим за x_k вершину Декартова дерева, содержащую k -тый по порядку ключ.
 - (a) Пусть $1 \leq i, k \leq n$. Найдите вероятность того, что x_k — предок x_i .
 - (b) Найдите матожидание количества листьев дерева (точно, а не асимптотически).
 - (c) Пусть $1 \leq i, j, k \leq n$. Найдите вероятность того, что x_j — общий предок x_i и x_k .
Подсказка: общий предок — не обязательно LCA.
 - (d) Пусть $1 \leq i \leq k \leq n$. Найдите матожидание длины пути между x_i и x_k . Ответ достаточно выразить через комбинацию гармонических чисел $H_n = \sum_{d=1}^n \frac{1}{d}$.
2. Есть двоичное дерево построенное на множестве ключей K . Пусть $\text{splay}(k)$ для $k \in K$ — splay вершины, содержащей ключ k , и $\text{insert}(x)$ для $x \notin K$ — обычное добавление x в двоичное дерево (т.е. просто подвешиваем лист с x в единственное подходящее место).
 - (a) Пусть $|K| = 3$. Предъявите пару двоичных деревьев на K первое из которых нельзя получить из второго последовательностью операций splay .
 - (b) Пусть $|K| = 4$. Покажите, что любые два двоичных дерева на K можно получить друг из друга последовательностью операций splay .
 - (c) Пусть $k_{i-2}, k_{i-1}, k_i, k_{i+1} \in K$ — 4 последовательных ключа. Покажите, что выполнение splay для последовательности $k_{i-2}, k_{i-1}, k_i, k_{i+1}, k_{i-2}$ делает k_i листом.
 - (d) Пусть $k \in K$ и $x \notin K$. Покажите, что $\text{splay}(k)$ и $\text{insert}(x)$ коммутируют.
 - (e) Пусть $|K| \geq 4$. Покажите, что любые два двоичных дерева на K можно получить друг из друга последовательностью операций splay .
3. Даны число K и изначально пустая последовательность. Вам поступает n запросов, каждый одного из двух типов:
 - $\text{append}(x)$ — дописать элемент x в конец последовательности
 - $\text{rev}()$ — развернуть K последних элементов последовательности (если в данный момент их всего меньше K , то развернуть всю последовательность).

Вам нужно один раз после всех запросов вывести получившуюся последовательность.

- (a) $\mathcal{O}(n \log n)$
- (b) $\mathcal{O}(n)$

Дополнительные задачи

4. C-c-c-combo!

Напишите на `Haskell` реализацию персистентной структуры, умеющей отвечать на запросы:

- $\text{make}(x_0, x_1, x_2, \dots, x_{n-1})$ — создать структуру со значениями.
- $\text{insert}(i, x)$, $0 \leq i \leq L$ (L — текущая длина) — вставить x в позицию i . Все элементы в i и правее сдвигаются на 1 вправо.
- $\text{delete}(i)$ — удалить элемент из позиции i . Все правее сдвигается на 1 влево.
- $\text{add}(l, r, \text{value})$ — добавить value ко всем x , для которых $l \leq i \leq r$
- $\text{set}(l, r, \text{value})$ — установить в value все x , для которых $l \leq i \leq r$
- $\text{sum}(l, r)$ — сумма всех x , для которых $l \leq i \leq r$
- $\text{reverse}(l, r)$ — изменить порядок всех x , для которых $l \leq i \leq r$, на обратный

В начальный момент структура пустая. Время работы на запрос $\mathcal{O}(\log L)$, online.

5 Деревья и отрезки

5.1 Практика

1. Вывести все числа $\geq Y$ на отрезке $[L, R]$ массива. Static online.
 - (a) $\langle \mathcal{O}(n \log n), \mathcal{O}(\log^2 n + k) \rangle$ (k — размер ответа).
 - (b) $\langle \mathcal{O}(n \log n), \mathcal{O}(\log n + k) \rangle$.
 - (c) Определите k за $\langle \mathcal{O}(n \log n), \mathcal{O}(\log n) \rangle$ (разными способами).
2.
 - (a) `sum[l..r]` за $\mathcal{O}(\sqrt{n})$, `a[i] = x` за $\mathcal{O}(1)$.
 - (b) `sum[l..r]` за $\mathcal{O}(1)$, `a[i] = x` за $\mathcal{O}(\sqrt{n})$.
 - (c) Улучшите (a) до $\langle \mathcal{O}(n^{1/3}), \mathcal{O}(1) \rangle$.
 - (d) Улучшите (a) и (b) до $\langle \mathcal{O}(k \cdot n^{1/k}), \mathcal{O}(k) \rangle$.
3. Вам дана последовательность из n чисел. Нужно offline ответить на m запросов вида «количество различных чисел на отрезке l, r » за $\mathcal{O}((m+n)\sqrt{n})$.
4. Даны отрезки на прямой $[L_i, R_i]$, у каждого есть вес w_i .
 - (a) Запрос `get(x)` — находить отрезок max веса, покрывающий точку x . $\mathcal{O}(\log n)$, online.
 - (b) Запрос `get(x, m)` — то же, но среди отрезков не длиннее m . $\mathcal{O}(\log n)$, online.
5. Дан массив из n элементов, можно сделать предобработку за $\mathcal{O}(n \log n)$. Запросы: количество различных чисел на отрезке $[L, R]$. Тут будет подсказка про `prev[i]`.
 - (a) online за $\mathcal{O}(\log^2 n)$.
 - (b) offline за $\mathcal{O}(\log n)$.
 - (c) online за $\mathcal{O}(\log n)$.
6. Дано дерево, на вершинах которого могут быть пометки. Запросы: пометить вершину, снять пометку с вершины, число помеченных вершин в поддереве. $\langle \mathcal{O}(n), \mathcal{O}(\log n) \rangle$, online.
7. Дано дерево с весами на ребрах. Запросы:
 - Изменить вес ребра e .
 - Вывести вес простого пути между данной парой вершин. $\langle \mathcal{O}(n), \mathcal{O}(\log n) \rangle$, online.
8. Дано дерево с весами на ребрах. Нужно научиться отвечать на запрос `count(a, b, x)` — количество ребер на пути из a в b , у которых вес $\geq x$, если:
 - (a) Веса не меняются. $\mathcal{O}(\log n)$, online.
 - (b) Теперь веса меняются. Запрос за $\mathcal{O}(\log^2 n)$, изменение веса за $\mathcal{O}(\log^2 n)$. Online.

Дополнительные задачи

9. Запросы: k -е по порядку среди различных чисел на отрезке $[L, R]$.
 - (a) offline за $\mathcal{O}(\log^3 n)$.
 - (b) online за $\mathcal{O}(\log^3 n)$.
 - (c) online за $\mathcal{O}(\log^2 n)$.
10. Даны два дерева из n вершин, вершины в обоих помечены числами от 1 до n без повторений. Запрос: по вершине из первого дерева и вершине из второго дерева сказать количество чисел от 1 до n , которые лежат в поддеревьях обеих вершин из запроса. $\langle \mathcal{O}(n \log n), \mathcal{O}(\log n) \rangle$.

5.2 Домашнее задание

1. На плоскости даны n точек и m прямоугольников с целочисленными координатами. Стороны прямоугольников параллельны осям координат. Найдите количество точек, покрытых ровно k прямоугольниками. $\mathcal{O}((n + m) \log m)$.
2. Дана последовательность чисел длины n . Придумайте как за $\mathcal{O}(n \log n + m\sqrt{n} \log n)$ ответить на m следующих запросов:
 - `sum(l, r)` – посчитать сумму на отрезке $[l, r]$.
 - `inc(l, r, x)` – увеличить все значения отрезка $[l, r]$, меньшие x , до x .
3. Дана последовательность длины n . Элементы последовательности — целые числа, не превосходящие C . Научитесь обрабатывать online-запросы:
 - Циклически сдвинуть подотрезок последовательности на 1 влево или вправо. Например, в последовательности (1, 2, 3, 4, 5) циклический сдвиг (2, 3, 4) на 1 вправо даст последовательность (1, 4, 2, 3, 5)
 - Посчитать, сколько раз встречается число x на подотрезке последовательности. $\mathcal{O}(\sqrt{n})$ на запрос, предподсчёт за $\mathcal{O}(C\sqrt{n} + n)$.
4. Дана последовательность из n чисел и m отрезков $[l_i, r_i]$ на ней. Про каждый отрезок надо определить число инверсий на нем. $\mathcal{O}((m + n)\sqrt{n} \log n)$.

Дополнительные задачи

5. Даны натуральные числа $a_1, \dots, a_n$ и k , не превосходящие C . За $\mathcal{O}(C + (m + n)\sqrt{n})$ ответить на m запросов вида: по данным l, r найти количество таких i и j , что $l \leq i \leq j \leq r$ и *xor* чисел $a_i, a_{i+1}, \dots, a_j$ равен k .

6 Хеширование строк

6.1 Практика

Большинство алгоритмов в этой серии задач вероятностные, то есть могут ошибаться с вероятностью, не превышающей ε .

1. Придумайте для строки длины n предподсчет за $\mathcal{O}(n)$, позволяющий:
 - (a) Вычислить за $\mathcal{O}(1)$ полиномиальный хеш любой подстроки.
 - (b) Лексикографически сравнить любые две подстроки за $\mathcal{O}(\log n)$.
2. Пусть есть семейство полиномиальных хеш-функций $\mathcal{H}$ по простому модулю m со случайной точкой x . Покажите, что для любой пары строчек s_1, s_2 длины не более n :

$$\Pr_{h \in \mathcal{H}} [h(s_1) = h(s_2)] \leq \frac{n}{m}$$

3. Найдите все периоды строки. $\mathcal{O}(n)$.
4.
 - (a) Найти число различных подстрок в строке. $\mathcal{O}(n^2)$.
 - (b) Найти подстроку данной строки, встречающуюся максимальное число раз.
5. Найдите k -й в лексикографическом порядке суффикс в строке.
 - (a) $\mathcal{O}(n \log^2 n)$.
 - (b) $\mathcal{O}(n \log n)$.
6.
 - (a) Найдите позицию строки в ее суффиксном массиве. $\mathcal{O}(n \log n)$.
 - (b) Постройте суффиксный массив строки за $\mathcal{O}(n \log^2 n)$. Что оптимальнее использовать: `sort` или `stable_sort`?
7. Определим следующую последовательность строк:
 - $S_0 = 0$
 - $S_1 = 01$
 - $S_2 = 0110$
 - $S_n = S_{n-1}(\neg S_{n-1})$

Заметим, что каждая строка является префиксом всех следующих. Обозначим как S_∞ строку, содержащую каждую S_i как префикс. Докажите, что если результат полиномиального хеширования вычисляется по модулю 2^{64} , то вне зависимости от выбранной (нечетной) точки $\text{hash}(S_{11}) = \text{hash}(\neg S_{11})$.

8. Научиться искать образец в строке, если допустимо различие в один символ между образцом и найденной подстрокой. $\mathcal{O}(n \log m + m)$.
9. Найти наибольшую по длине строку, которая дважды без перекрытий встречается в заданной строке. $\mathcal{O}(n \log n)$.
10.
 - (a) Найти количество подпалиндромов строки. $\mathcal{O}(n \log n)$.
 - (b) Найти максимальный подпалиндром строки. $\mathcal{O}(n)$.

Дополнительные задачи

11. Дано начальное состояние в игре «Жизнь» в виде битовой матрицы $N \times N$. Нас интересует, сколько останется живых клеточек на поле после большого количества итераций $T \gg N$. Для простоты можно считать N и T степенями двойки.
- (а) Определим функцию e , сопоставляющую тайлу размера $2^n \times 2^n$ кусок размера $2^{n-1} \times 2^{n-1}$ из его середины на 2^{n-2} шагов в будущем. Можно ли написать рекуррентное соотношение для e ?
 - (б) Придумайте, как с помощью хеширования решить нашу задачу быстрее, чем наивной симуляцией.

6.2 Домашнее задание

Определение. Семейство хэш-функций $\mathcal{H} = \{h : X \rightarrow Y\}$ называется

- универсальным, если:

$$\forall x_1, x_2 \in X, x_1 \neq x_2 : \Pr_{h \in \mathcal{H}} [h(x_1) = h(x_2)] \leq \frac{1}{|Y|}$$

- k -независимым, если для любых различных $x_1, x_2, \dots, x_k \in X$, для любых, возможно, совпадающих $y_1, y_2, \dots, y_k \in Y$ выполняется:

$$\Pr_{h \in \mathcal{H}} \left[\bigwedge_{i=1}^k h(x_i) = y_i \right] = \frac{1}{|Y|^k}$$

1. (a) Докажите, что любое 2-независимое семейство хэш-функций является универсальным.
(b) Докажите, что любое $k+1$ -независимое семейство хэш-функций является k -независимым.
2. Придумайте строку длины n над алфавитом $\{0, 1\}$, в которой $\Omega(n^2)$ различных подстрок.
3. Даны две строки суммарной длины n . Найдите их самую длинную общую подстроку. $\mathcal{O}(n \log n)$.
4. Даны строка s длины n и число k . Найдите в s за время $\mathcal{O}(n)$ самую длинную подстроку, представимую как степень некоторой строки x , для которой $|x| = k$ (степенью строки называют её конкатенацию с собой несколько раз).
5. Найти подстроку в тексте. При сравнении строк, если несовпадений символов было не более k , строки считаются равными. $\mathcal{O}(nk \log n)$.
6. Дана строка длины n . Запросы:
 - `set(i, c)` – установить символ на i -й позиции в s .
 - `test(i, j, l)` – проверить на равенство подстроки длины l начинающиеся в i -й и j -й позициях.

$\mathcal{O}(\log n)$ на запрос, online.

Дополнительные задачи

7. Напишите на `Haskell` определение бесконечного списка `Int`-ов (из $\{0, 1\}$), представляющего собой последовательность Туэ-Морса (последовательность из задачи 7 практики) S_∞ , такое, что:
 - определение содержит не более 128 символов
 - можно использовать стандартные функции из модулей `Prelude` и `Data.List`
 - вычисление в нормальную форму `genericTake n` от списка занимает время $\mathcal{O}(n)$.
8. Пусть дана w -разрядная `RAM` машина. Для простоты можно считать w степенью двойки. Придумайте для такой машины детерминированный предподсчет за $\mathcal{O}(w)$ операций, который позволит по w -битному числу вида 2^k восстанавливать k за $\mathcal{O}(1)$ операций.

7 Универсальные семейства

7.1 Практика

1. Является ли семейство из одной совершенной хеш-функции:
 - (a) универсальным?
 - (b) 2-независимым?
2. Сколько существует различных функций из $\mathbb{F}_2^n$ в $\mathbb{F}_2^m$?
3. Пусть случайно равномерно выбрана функция $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$. Найдите вероятность события
 - (a) $f(x) = y$ для фиксированных x и y ,
 - (b) $f(x_1) = f(x_2)$ для фиксированных разных x_1 и x_2 ,
 - (c) $f(x_1)$ отличается от $f(x_2)$ ровно в одном бите.
4.
 - (a) Для каких n и m семейство всех функций $\mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ является универсальным?
 - (b) ... 2-независимым?
 - (c) Может ли существовать конечное универсальное семейство функций $\{f : X \rightarrow \mathbb{F}_2^m\}$, если X бесконечно?
5. В вашем распоряжении есть пара 2-независимых семейств хеш-функций $\mathcal{A} = \{f : A \rightarrow \mathbb{F}_2^n\}$ и $\mathcal{B} = \{g : B \rightarrow \mathbb{F}_2^n\}$. Постройте (и докажите, что построено правильно) 2-независимое семейство хеш-функций:
 - (a) $\{h : A \cup B \rightarrow \mathbb{F}_2^n\}$ при условии $A \cap B = \emptyset$,
 - (b) $\{h : A \times B \rightarrow \mathbb{F}_2^n\}$.
6. Построим хеш-функцию $h : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^k$ для $k < n$. Зафиксируем матрицу A ранга k над полем $\mathbb{F}_2$ и вектор $b \in \mathbb{F}_2^k$. Пусть
$$h(x) := A \cdot x + b.$$
 - (a) Найдите I максимальной мощности такое, что $|h(I)| = 1$. Какова мощность I ?
 - (b) Постройте (и докажите, что построено правильно) 2-независимое семейство хеш-функций $\mathcal{H} = \{h_i : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^k\}_i$.
7. Пусть p — простое. Определим семейство $\mathcal{H} = \{f : \mathbb{F}_p^n \rightarrow \mathbb{F}_p\}$ следующим образом:

$$f(x) = b + \sum_k a_k x_k \pmod p$$

где b и a_k выбираются равномерно независимо из $[0 \dots p-1]$. Покажите, что получилось 2-независимое семейство.

8. Пусть p — простое. Определим хеш-функцию $h_{a_0, \dots, a_{k-1}}(x) := \sum_{i=0}^{k-1} a_i \cdot x^i \pmod p$. Покажите, что семейство $\mathcal{H} = \{h_{a_0, \dots, a_{k-1}} \mid a_i \in \mathbb{Z}_p\}$ является k -независимым.

Дополнительные задачи

9. Даны два множества A и B , на которых построены фильтры Блума $\mathcal{A}$ и $\mathcal{B}$ соответственно с одинаковыми размерами и хеш-функциями.
 - (a) Правда ли, что фильтр Блума, построенный на $A \cup B$ совпадает с $\mathcal{A} \cup \mathcal{B}$?
 - (b) Правда ли, что фильтр Блума, построенный на $A \cap B$ совпадает с $\mathcal{A} \cap \mathcal{B}$?
 - (c) Правда ли, что фильтр Блума $\mathcal{A} \cap \mathcal{B}$ отвечает положительно на все элементы $A \cap B$?
10. Придумайте способ эффективно удалять ранее добавленные элементы из фильтра Блума (вероятно, потребуется больше в $\log n$ раз памяти).

7.2 Домашнее задание

1. Пусть A — непустое конечное множество и $\mathcal{A} = \{f : A \rightarrow \mathbb{F}_2^n\}$ — 2-независимое семейство хеш-функций на нем. Обозначим за L_k множество списков элементов из A длины **ровно** k , за $L_{\leq k}$ — длины **не более** k и за L — списков любой длины.

- (a) Фиксируем k и введем семейство хеш-функций $\mathcal{H} = \{h : L_k \rightarrow \mathbb{F}_2^n\}$ следующим образом: пусть $c \in \mathbb{F}_2^n$ и $f_0, f_1, \dots, f_{k-1} \in \mathcal{A}$ семплированы случайно независимо. Тогда:

$$h([l_0, l_1, \dots, l_{k-1}]) = c \oplus f_0(l_0) \oplus f_1(l_1) \oplus \dots \oplus f_{k-1}(l_{k-1})$$

Покажите, что полученное семейство 2-независимое.

Подсказка: используйте результат с практики и индукцию по k .

- (b) Теперь аналогично определенное семейство хеш-функций $\mathcal{H} = \{h : L_{\leq k} \rightarrow \mathbb{F}_2^n\}$: независимо семплируем $c \in \mathbb{F}_2^n$ и $f_0, f_1, \dots, f_{k-1} \in \mathcal{A}$, и:

$$h(l) = c \oplus f_0(l_0) \oplus f_1(l_1) \oplus \dots \oplus f_{len(l)-1}(l_{len(l)-1})$$

(тут $len(l) \leq k$). Покажите, что полученное семейство также 2-независимое.

Подсказка: не забудьте про пустой список.

- (c) Придумайте 2-независимое семейство $\{L \rightarrow \mathbb{F}_2^n\}$. Если оно получится бесконечным, также придумайте практический способ как им пользоваться.
2. Равномерной называется хеш-функция, у которой у каждого значения одинаковое количество прообразов. Пусть есть пара множеств A, B и две равномерные хеш-функции $f : A \rightarrow [0 \dots n-1]$ и $g : B \rightarrow [0 \dots n-1]$, где n — натуральное число. Постройте равномерную хеш-функцию с областью значений $[0 \dots n-1]$ для:
 - (a) Упорядоченных пар (A, B) ,
 - (b) Неупорядоченных пар $\{A, A\}$, если n — нечётное.
 - (c) Покажите, что при четном n соответствующей хеш-функции для $\{A, A\}$ может не существовать.
 3. Есть семейство хеш-функций $\mathcal{H} = \{h : K \rightarrow Y\}$ и подмножество $X \subseteq K$, $|X| > 1$. Покажите, что, если $\mathcal{H}$ универсальное и $|Y| = c|X|^2$, то

$$\Pr_{h \in \mathcal{H}} [|h(X)| = |X|] \geq 1 - \frac{1}{2c}$$

Подсказка: вам может пригодиться неравенство Маркова.

8 Алгебра

8.1 Практика

1. Дано число n , a и b . Известно, что $a^2 = b^2 \pmod{n}$, но $a \not\equiv \pm b \pmod{n}$. Найдите нетривиальное разложение n на множители.
2. (а) Заметим, что $\gcd(2x, 2y) = 2\gcd(x, y)$ и, если x — нечетное число, то $\gcd(x, 2y) = \gcd(x, y)$. Придумайте $\gcd$ за $\mathcal{O}(n^2)$.
(б) Придумайте нерекурсивную версию расширенного Евклида.
(в) Докажите, что для тождества Безу $Ax + By = \gcd(x, y)$, которое вернет расширенный Евклид, если $x, y > 0$, то $|A| \leq y$ и $|B| \leq x$.
3. Дана матрица A размера $n \times m$ над конечным полем, $n \geq m$. Найдите матрицу B размера $m \times n$ такую, что $BA = I_{m \times m}$, или установите, что такой не существует.
4. Перед нами стоит задача: получить случайное простое число в диапазоне от 1 до n . Решать ее будем так: выбираем случайное число равномерно из диапазона и запускаем тест Миллера-Рабина k раз. Если все k тестов прошли, то возвращаем число, иначе — повторяем весь процесс сначала. Этот метод не так плох как кажется, т.к. простые числа достаточно часты (среди чисел от 1 до n примерно $\frac{n}{\ln n}$ простых чисел). Известно, что любое составное число проваливает тест Миллера-Рабина с вероятностью $\frac{3}{4}$, и повторные запуски теста независимы друг от друга. Какое минимальное k нужно выбрать, чтобы получить простое число с вероятностью $\geq 90\%$?
5. Дано множество натуральных чисел A и параметр b . Известно, что каждое число из A раскладывается в произведение степеней первых b простых чисел (т.е. $\forall a \in A \ a = \prod_{i=1}^b p_i^{\alpha_i}$). Требуется найти непустое подмножество A , числа из которого в произведении дадут квадрат некоторого натурального числа. Придумайте полиномиальный от $|A|$, $\log \max A$ и b алгоритм, для поиска такого подмножества.
6. Есть множество $A \subseteq [n]$ и k ячеек, каждая из которых умеет хранить $\mathcal{O}(\log n)$ бит информации. В каждый момент времени максимум t из k ячеек могут оказаться недоступны (мы можем узнать про ячейку, доступна ли она, и, если доступна, прочитать данные). Требуется организовать такой способ хранения информации (стирающий код), чтобы в любой момент времени можно было восстановить множество A .
(а) $k = (t + 1) \cdot |A|$.
(б) $k = t + |A|$.
7. Вам дана система линейных уравнений по модулю n . Решите данную систему при условии, что:
(а) $n = pq$, p, q — разные простые.
(б) $n = p^k$, p — простое.
(в) для произвольного n .
8. В d -мерном пространстве над $\mathbb{Q}$ даны точка и набор из k векторов — базис подпространства. Найдите расстояние от точки до подпространства.
9. Для заданных n , k и простого p посчитайте за линейное время $\binom{n}{k} \pmod{p}$. Учтите, что p может быть меньше, чем n . Можно считать, что все операции в $\mathbb{Z}_p$ выполняются за $\mathcal{O}(1)$.

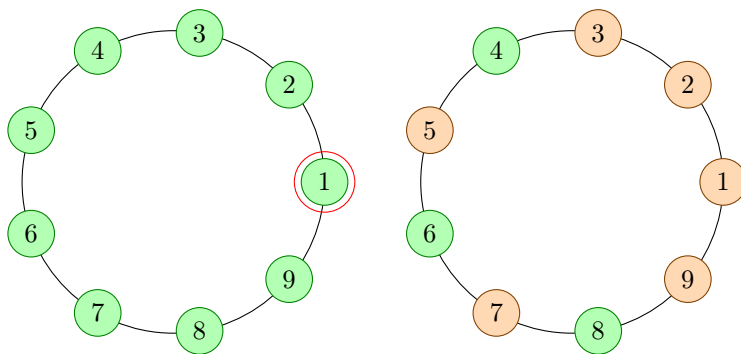
Дополнительные задачи

10. Дан набор векторов с весами, оболочка векторов совпадает со всем пространством. Выбрать из данных векторов базис минимального суммарного веса.
11. Дана матрица A размера 2×2 над кольцом целых чисел. Придумайте алгоритм, представляющий A в виде $A = LDR$, где D — диагональная матрица, а L и R — обратимые.

8.2 Домашнее задание

1. Придумайте расширенную версию двоичного `gcd` из задачи 2а практики.
2. Дана матрица A размера $n \times m$. Найти такое представление $A = B \cdot C$, что B имеет размер $n \times k$ и k минимально. $\mathcal{O}(nm(n+m))$.
Подсказка: $k = \text{rank}(A)$.
3. Лягушонок Вася живёт на прямой. Устройство лапок позволяет ему прыгать только на $a \in \mathbb{N}$ или $b \in \mathbb{N}$ **вперёд**. Может ли он допрыгать из точки 0 в точку c ? $\mathcal{O}(\text{poly}(\log(a+b)))$.
4. На кольцевой стоят n светофоров и хитро перемигиваются. Свет на каждом светофоре зажигается в таком порядке: зелёный, желтый, красный, снова зелёный и так по кругу. Все светофоры пронумерованы от 1 до n . Если переключить светофор с номером i , то следующие k_i светофоров с шагом a_i тоже автоматически поменяют свой цвет. (Ниже приведен пример, когда переключается первый светофор, $k_1 = 5$, $a_1 = 2$.)

Приведите алгоритм, который по заданному начальному состоянию светофоров и всем k_i , a_i за время $\mathcal{O}(n^3)$ определяет, возможен ли «зелёный коридор» — ситуация, когда все светофоры переключены на зелёный.



5. У Винни-Пуха есть бесконечные запасы каждого из k типов горшочков мёда. Каждый тип горшочков характеризуется целым числом — сколько дней нужно Винни-Пуху, чтобы съесть весь мёд из одного такого горшка. Винни-Пух уже провёл серию экспериментов вида “взять несколько горшочков мёда, записать день недели, когда эксперимент начался, есть мёд, пока тот не закончится, записать день недели, когда эксперимент закончился” (в наборе может быть несколько горшочков одного типа, в неделе 7 дней). По понятным причинам Винни очень любит экспериментировать. А Кролик не любит мёд, но любит предсказывать будущее. Помогите Кролику определить, можно ли, зная результаты первых k экспериментов и набор горшков и день начала для $(k+1)$ -го, предсказать день недели, когда $(k+1)$ -й эксперимент закончится.
 - (а) Для каждого эксперимента Винни берёт произвольные наборы горшков. $\mathcal{O}(k^3)$
 - (б) Для каждого эксперимента Винни берёт горшки не более чем двух разных типов. $\mathcal{O}(k)$.

Дополнительные задачи

6. Рассмотрим матрицу $A \in \{0,1\}^{n \times m}$. Для произвольных i и j ($1 \leq i \leq n$, $1 \leq j \leq m$) разрешается поменять все значения в строке i и столбце j на противоположные (значение на пересечении строки и столбца меняется). Требуется получить нулевую или единичную матрицу. Придумайте алгоритм за $\mathcal{O}((nm)^3)$.
7. В распоряжении взломщиков оказался волшебный оракул, способный для любого открытого ключа (n, e) взломать 1% из возможных зашифрованных протоколом RSA сообщений (т.е.

детерминированная функция, которая за $\mathcal{O}(1)$ успешно сопоставляет набору (n, e, y) такое число m , что $y \equiv m^e \pmod{n}$, на случайно выбранной сотой части всех возможных входов). Придумайте алгоритм, который взламывает любое сообщение со средним временем работы $\mathcal{O}(\text{poly}(\log n))$.

8. Для заданного n и простого p найти число таких $k \in [0, n]$, что $\binom{n}{k} \equiv 0 \pmod{p}$. Можно считать, что все операции по модулю p происходят за $\mathcal{O}(1)$. Придумайте алгоритм, который работает за $\mathcal{O}(\text{poly}(\log n))$ (Подсказка: докажите, что $(1+x)^{p^i} \equiv 1+x^{p^i} \pmod{p}$, представьте числа в p -ичной системе счисления и попробуйте упростить $(x+1)^n$)

9 FFT

9.1 Практика

1. Пусть вам известно $\text{DFT}([a_0, a_1, \dots, a_{n-1}])$. Вычислите $\text{DFT}([a_1, a_2, \dots, a_n])$ за $\mathcal{O}(n)$.
2. Вам даны две последовательности a_i и b_i длины n . Вычислите последовательность c_k :
 - (a) $c_k = \sum_{i=0}^{n-1} a_i b_{(k-i) \bmod n}$, если n — степень двойки
 - (b) $c_k = \sum_{i=0}^{n-1} a_i b_{(k-i) \bmod n}$
 - (c) $c_k = \sum_{i=0}^k a_i b_{k-i}$
 - (d) $c_k = \sum_{i=0}^{n-1} a_i b_{(k+i) \bmod n}$ $\mathcal{O}(n \log n)$.
3. За какое время можно посчитать 2^n в десятичной системе счисления, используя FFT?
4. Даны текст t и шаблон p над алфавитом размера k . $|t| \geq |p|$.
 - (a) Для каждого из $|t| - |p| + 1$ наложений p на t узнать количество ошибок. $\mathcal{O}(k|t| \log |t|)$.
 - (b) Предыдущий пункт с дополнительным условием: в тексте и в шаблоне допустимы символы «?», означающие совпадение с любым символом.
5. Покажите, что, если к массиву длины n последовательно применить FFT 4 раза, то получится исходный массив умноженный на n^2 .
6. Дано уравнение: $x^n + y^n \equiv z^n \pmod{m}$. Найдите количество его решений за $\mathcal{O}(m \log(n + m))$.
7. Эльф Леголас — пессимист. Начиная со своего сотого дня рождения каждому дню своей жизни он присваивает рейтинг уныния — действительное число от 0 до 1. Жизнь Леголаса была печальна, но насыщена: из года в год одни неприятности уступали место другим. Однако последние k лет у Леголаса появилось стойкое ощущение Déjà vu: ему кажется, что происходящие с ним невзгоды сильно напоминают один из предыдущих отрезков его страдальческой жизни, — но вот какой именно, он вспомнить не может. Воспользовавшись дневником рейтинга уныния, помогите Леголасу найти такой период в его жизни, что среднеквадратичное отличие рейтинга уныния каждого дня от соответствующего дня последних k лет минимально. Время $\mathcal{O}(N \log N)$, где N — возраст Леголаса.
8. Дана чёрно-белая (без оттенков серого) картинка размера $n \times n$ и образец размера $k \times k$, $k \leq n$. Найдите наилучшее вхождение образца в картинку за $\mathcal{O}(n^2 \log n)$. Наилучшим называется вхождение с минимальным суммарным количеством различных пикселей.

Дополнительные задачи

9. (a) Пусть дан многочлен $p(x)$. Определим многочлен $R(p(x))$ как «разворот» $p(x)$: коэффициент при константе меняется с коэффициентом при $x^{\deg(p(x))}$, коэффициент при x меняется с коэффициентом при $x^{\deg(p(x))-1}$, и так далее. Докажите, что $R(p_1(x)p_2(x)) = R(p_1(x))R(p_2(x))$.
(b) Пусть p_1 при делении на p_2 дает неполное частное q и остаток r . Докажите, что $R(p_1) \equiv R(p_2)R(q) \pmod{x^{\deg(p_1)-\deg(p_2)}}$.
(c) Придумайте, как зная $R(p_2)^{-1} \bmod x^k$ найти $R(p_2)^{-1} \bmod x^{2k}$ за $\mathcal{O}(1)$ умножений многочленов.
(d) Придумайте алгоритм, реализующий деление многочленов степени не более n с остатком за $\mathcal{O}(n \log n)$.
10. Даны n предметов и запросы «можно ли набрать вес w_i , используя только предметы с номерами от l_i до r_i ». При этом все $w_i \leq s$. Сделайте предподсчёт за $\mathcal{O}(ns \log s)$ так, чтобы на запрос можно было бы в online ответить за $\mathcal{O}(s \log s \log n)$.

9.2 Домашнее задание

- Пусть p — простое число и g — первообразный корень по модулю p . Определим DFT размера p обычным образом:

$$f_k = \sum_{i=0}^{p-1} x_i \omega_p^{ki}$$

- Определим $x'_s = x_{g^{-s} \bmod p}$, $f'_r = f_{g^r \bmod p}$ и $\Delta_i = \omega_p^{g^i}$. Покажите, что:

$$f'_r = x_0 + \sum_{s=0}^{p-2} x'_s \Delta_{(r-s) \bmod (p-1)}$$

- Придумайте, как вычислить f_k за $\mathcal{O}(p \log p)$ арифметических операций.
- Придумайте, как свести вычисление DFT последовательности размера $n = n_1 n_2$ к:
 - n_1 вычислениям DFT от последовательностей размера n_2 и $\mathcal{O}(n_1^2 n_2)$ дополнительных арифметических операций.
 - n_1 вычислениям DFT от последовательностей размера n_2 , n_2 вычислениям DFT от последовательностей размера n_1 и $\mathcal{O}(n_1 n_2)$ дополнительных арифметических операций.
 - Вам даны две последовательности a_i и b_i длины $n = 2^m$. Вычислите за $\mathcal{O}(n \log n)$ арифметических операций последовательность:

$$c_k = \sum_{i \oplus j = k} a_i b_j$$

Вам может пригодиться отображение $x_i \rightarrow f_i$ определенное следующим образом:

$$f_k = \sum_{i=0}^{n-1} x_i (-1)^{\text{bit_count}(k \wedge i)}$$

Подсказка: покажите, что:

$$(-1)^{\text{bit_count}(k \wedge (i \oplus j))} = (-1)^{\text{bit_count}(k \wedge i)} \cdot (-1)^{\text{bit_count}(k \wedge j)}$$

- Найдите количество AVL-деревьев высоты h из n вершин по простому модулю p за:
 - $\mathcal{O}(hn \log n)$.
 - $\mathcal{O}(hN(h))$. Здесь $N(h)$ — максимальное количество вершин в AVL дереве высоты h .

Считайте, что в поле вычетов по модулю p вам известен первообразный корень из единицы достаточной степени.
- Даны строка s длины n и шаблон p длины $m \leq n$. Алфавит — **не** константа.
 - Найдите точные вхождения шаблона за $\mathcal{O}(1)$ DFT размера порядка n .
Подсказка: посчитайте $\sum_{j=0}^{m-1} (s_{i+j} - p_j)^2$ для всех $0 \leq i \leq n - m$.
 - Теперь шаблон может содержать вопросительные знаки. Найдите подстроку s , точно подходящую под шаблон. $\mathcal{O}(n \log n)$.
 - Шаблон содержит звёздочки и вопросительные знаки. Проверьте, подходит ли строка **целиком** под шаблон. $\mathcal{O}(n\sqrt{n} \log n)$.

10 Потоки и разрезы

10.1 Практика

1. Дан двудольный граф. Каждой вершине сопоставлено число a_v .
 - (a) Выберите максимальное количество рёбер так, чтобы степени вершин были не более 1.
 - (b) Выберите максимальное количество рёбер так, чтобы степени вершин были не более a_v .
2. Даны девочки, мальчики и собачки. Для каждой пары “мальчик, девочка” известно, хочет ли девочка дружить с мальчиком. Для каждой пары “собачка, девочка” известно, нравится ли собачка девочке. Нужно максимальному количеству девочек выделить по мальчику и собачке так, что:
 - Каждый мальчик не более чем с одной девочкой.
 - Каждая собачка не более чем у одной девочки.
 - Тройки гармоничны: девочка и хочет дружить с выбранным ей мальчиком, и собачка ей нравится.
3. Дан неориентированный граф. Необходимо ориентировать его так, чтобы максимальная исходящая степень была минимальна.
 - (a) $\mathcal{O}(E^2 \log V)$
 - (b) $\mathcal{O}(E^2)$
4. По правилам футбольного турнира в каждом матче должна победить одна из команд, то есть, не бывает ‘ничьих’. Вам дана матрица уже сыгранных матчей. Можно ли так доиграть турнир, чтобы каждая команда выиграла заданное число раз? Каждая команда играет с каждой, для каждой команды известно, сколько игр она выиграла.
5. Каждой вершине ориентированного графа сопоставлено число (не обязательно положительное) — её вес. Найдите замкнутое подмножество вершин максимальной суммарной стоимости. Подмножество вершин называется замкнутым, если из него не исходят рёбра в другую часть графа.
6. Рассмотрим ориентированный граф. За одно действие можно удалить все входящие в вершину i ребра за стоимость $a_i \geq 0$, или все исходящие из вершины i рёбра за стоимость $b_i \geq 0$. Необходимо удалить все рёбра графа за минимальную стоимость.
7. Есть ориентированный граф с начальной и конечной вершинами. В начальной вершине есть K грузовиков. Грузовикам нужно попасть в конечную вершину. Время дискретно. За единицу времени каждый грузовик или стоит на месте, или перемещается в одну из соседних вершин. В любой вершине могут одновременно стоять несколько грузовиков. По любому из рёбер в каждый момент времени должен ехать не более чем один грузовик. Минимизируйте время, когда все грузовики окажутся в конечной вершине.
 - (a) $\mathcal{O}(\text{poly}(V, E, K))$
 - (b) $\mathcal{O}(K(V + K)E)$

Дополнительные задачи

8. Найдите максимальную по числу вершин двудольную клику (полный двудольный подграф) в двудольном графе за $\mathcal{O}(VE)$.
9. За один ход можно покрасить один произвольный вертикальный или горизонтальный отрезок матрицы $n \times t$ в белый цвет, мазки могут перекрываться. Приведите чёрную матрицу к заданному виду за минимальное число ходов. $\mathcal{O}(\text{poly}(n, t))$.

10.2 Домашнее задание

- (a) Разбейте вершины ориентированного графа на циклы. Т.е. каждая вершина должна быть покрыта ровно одним циклом. Либо скажите, что это невозможно. $\mathcal{O}(EV)$.
(b) Разбейте вершины ориентированного ациклического графа на минимальное количество путей. $\mathcal{O}(EV)$.
- Дан граф и выделенные вершины s, t . Нужно проверить, правда ли существует единственный минимальный s - t разрез.
(a) $\mathcal{O}(\text{poly}(V, E))$
(b) $\mathcal{O}(E)$ при условии, что нам уже известен максимальный поток (сам поток, не только его величина).

3. (кроме группы Мишунина)

Есть заказы и инструменты. Для каждого заказа известен список инструментов, который нужен, чтобы его выполнить. Каждый инструмент сделан умелыми японскими рабочими, поэтому бесконечно прочный, его можно один раз купить и много раз использовать. У каждого инструмента есть цена p_i . У каждого заказа есть прибыль, которую можно получить, выполнив заказ. Вы – бедный китайский рабочий. У вас изначально нет инструментов, но зато вы можете под нулевой процент в банке взять сколь угодно большой кредит, чтобы купить инструментов.

- (a) Вопрос: какую максимальную прибыль вы можете получить?
(b) А теперь тот же вопрос, но ещё есть разные скидочные предложения! Скидка позволяет два инструмента i, j купить по специальной цене d : $\max(p_i, p_j) < d < p_i + p_j$. Каждый инструмент присутствует не более чем в одном скидочном предложении.
- В неориентированном графе без кратных рёбер необходимо удалить минимальное число рёбер так, чтобы увеличилось количество компонент связности.
(a) $\mathcal{O}(V \cdot \text{Flow})$, где Flow — время работы какого-нибудь алгоритма поиска максимального потока.
(b) $\mathcal{O}(E^2)$.
- (только группа Мишунина)
Пусть целевая функция $f: \mathbb{F}_2^n \rightarrow \mathbb{R}$ определена как:

$$f(x_0, x_1, \dots, x_{n-1}) = \sum_{i=0}^{n-1} w_i(x_i) + \sum_{i < j} w_{ij}(x_i, x_j)$$

При этом для каждой w_{ij} выполняется $w_{ij}(0, 0) + w_{ij}(1, 1) \leq w_{ij}(0, 1) + w_{ij}(1, 0)$. Сведите минимизацию f к минимальному разрезу в графе полиномиального размера с неотрицательными весами.

Подсказка: попробуйте решить задачу отдельно для двух слагаемых и представить:

$$w_{ij}(x_i, x_j) = A_{ij} + B_{ij}x_i + C_{ij}x_j + D_{ij}\mathbb{I}[x_i \neq x_j]$$

Дополнительные задачи

- Пусть алгоритм Форда-Фалкерсона ищет дополняющие пути DFS-ом с фиксированным порядком просмотра рёбер в каждой вершине, и пропускает максимум по найденному дополняющему пути. Постройте вместе с порядком рёбер (или докажите, что такого не существует) ориентированный граф:
(a) с целочисленными пропускными способностями, на котором алгоритм работает за экспоненту от V (пропускные способности могут быть большими, но должны быть полиномиальной суммарной длины).

- (b) с вещественными пропускными способностями, на котором алгоритм не завершается за конечное количество шагов.
 - (c) с вещественными пропускными способностями, на котором алгоритм не сходится к корректному значению величины максимального потока.
7. Какое максимальное количество уголков можно разместить на шахматной доске $n \times m$ с дырками? Уголкем называется фигура, состоящая из трех клеток: центральная клетка черного цвета и две соседних с ней белых клетки со смежными сторонам.

11 Линейное программирование

11.1 Практика

1. Будем называть:

$$\begin{array}{ll} c^T x \rightarrow \max & c^T x \rightarrow \max \\ Ax \leq b & Ax = b \\ x \geq 0 & x \geq 0 \end{array}$$

стандартной формой и *канонической формой* задачи линейного программирования соответственно. Приведите задачу в стандартной форме к канонической и задачу в канонической форме к стандартной.

2. Сведите к задаче линейного программирования следующую задачу:

$$\begin{aligned} \min_{1 \leq i \leq p} \left[\sum_{j=1}^n c_{ij} x_j \right] &\rightarrow \max \\ \sum_{j=1}^n a_{ij} x_j &\leq b_i, i \in [1 \dots m] \\ x_i &\geq 0, i \in [1 \dots n] \end{aligned}$$

3. Пусть

$$\begin{array}{ll} c^T x \rightarrow \max & b^T y \rightarrow \min \\ Ax \leq b & A^T y \geq c \\ x \geq 0 & y \geq 0 \end{array}$$

пара из прямой и дуальной задач с оптимальными решениями x^* и y^* соответственно.

- (a) Покажите, что дуальной задачей к дуальной является прямая.
- (b) Постройте дуальную к задаче в канонической форме.
- (c) (weak duality) Покажите, что $c^T x \leq y^T Ax \leq b^T y$.
- (d) (complementary slackness) Покажите из strong duality, что условия $y_i^* > 0$ и $(Ax^*)_i < b_i$ никогда не выполняются вместе.
- (e) Покажите, что для задачи

$$\begin{array}{l} c^T x \rightarrow \max \\ Ax \leq b + \Delta b \\ x \geq 0 \end{array}$$

оптимальное значение целевой функции ограничено сверху $c^T x^* + \Delta b^T y^*$.

- (f) Покажите, что ровно одно из этих утверждений правда:

- i. $\exists x \geq 0: Ax \leq b$
- ii. $\exists y \geq 0: A^T y \geq 0 \wedge b^T y < 0$

Подсказка: используйте теорему о разделяющей гиперплоскости.

- (g) (strong duality) Покажите, что $c^T x^* = b^T y^*$.

4. Рассмотрим *матричную игру с нулевой суммой*: дана матрица A размера $n \times m$, первый игрок выбирает $i \in [1 \dots n]$, второй $j \in [1 \dots m]$. Выигрыш первого игрока составляет $A_{i,j}$, второго $-A_{i,j}$.

- (a) Пусть стратегия первого (второго) игрока — распределение вероятностей на строчках (столбцах). Первый игрок пытается максимизировать матожидание своего выигрыша в предположении, что второй игрок знает его стратегию. Сведите поиск оптимальной стратегии для первого игрока к задаче линейного программирования.
- (b) Аналогично для второго игрока.

- (с) Постройте дуальные к полученным задачам.
- (d) Покажите, что в данной игре существует *равновесие по Нэшу*. Пара стратегий a и b называется равновесием по Нэшу, если никому не выгодно отказываться от своей стратегии, то есть если первый игрок сменит стратегию a на c , а второй продолжит придерживаться стратегии b , то результат первого не улучшится; аналогично для второго.

Дополнительные задачи

5. Сведите к задаче линейного программирования задачу:

$$\frac{\sum c_i x_i + \alpha}{\sum d_i x_i + \beta} \rightarrow \min$$

$$\sum_{j=1}^n a_{ij} x_j = b_i, i \in [1 \dots m]$$

$$x_i \geq 0, i \in [1 \dots n]$$

Для простоты будем считать, что знаменатель целевой функции положителен в ограниченной области.

6. Пусть Вася умеет решать системы линейных неравенств из k уравнений от n неизвестных за время $LN(n, k)$. Васе дали задачу линейного программирования в стандартной форме с целочисленной матрицей A размера $n \times m$. Докажите, что он сумеет решить ее за $LN(n, m + 1)n \log(Mn)$, где M — максимальное число, встречающееся в матрице A , векторе или в минимизируемом функционале.
7. Рассмотрим задачу линейного программирования в канонической форме с матрицей A . Докажите, что точка x является вершиной полиэдра системы тогда и только тогда, когда столбцы матрицы A , индуцированные переменными, положительными в точке x , линейно независимы.

11.2 Домашнее задание

1. Сведите к задаче линейного программирования задачу:

$$\sum_{i=1}^p \left| \sum_{j=1}^n c_{ij} x_j - d_i \right| \rightarrow \min$$

При ограничениях:

$$\begin{aligned} \sum_{j=1}^n a_{ij} x_j &\leq b_i, i \in [1 \dots m] \\ x_i &\geq 0, i \in [1 \dots n] \end{aligned}$$

2. 9 вопросов: приведите пример (или докажите, что такого не бывает), когда прямая задача ЛП {несовместна, неограничена, имеет конечное решение} и дуальная задача {несовместна, неограничена, имеет конечное решение}.
3. Найдите равновесие Нэша (и докажите, что это действительно оно) в игре с нулевой суммой «камень-ножницы-бумага».
4. Пусть у нас задан орграф $G = (V, E)$ с двумя выделенными различными вершинами $s, t \in V$. Для каждого ребра $e \in E$ задано вещественное неотрицательное число c_e — его пропускная способность.

- (a) s - t потоком называется функция $f : E \rightarrow \mathbb{R}^+$ такая, что:

$$\forall e \in E : 0 \leq f_e \leq c_e$$

$$\forall v \in V \setminus \{s, t\} : \sum_{e=(u,v)} f_e - \sum_{e=(v,u)} f_e = 0$$

Величиной потока называется:

$$|f| = \sum_{e=(s,u)} f_e - \sum_{e=(u,s)} f_e$$

Сведите задачу нахождения s - t потока максимальной величины к задаче линейного программирования в стандартной форме.

Подсказка: добавить фиктивное ребро $t \rightarrow s$ и искать циркуляцию, максимизирующую поток по нему, немного упрощает решение следующих пунктов.

- (b) s - t разрезом называется пара $(S, T = V \setminus S)$ подмножеств V такая, что $s \in S$ и $t \in T$. Весом разреза называется величина:

$$\sum_{e=(u \in S, v \in T)} c_e$$

Сведите задачу о s - t разрезе минимального веса к задаче линейного программирования в стандартной форме.

Подсказка: назначьте каждой вершине v переменную $0 \leq y_v \leq 1$; покажите, что любое нецелочисленное решение представляется в виде выпуклой комбинации целочисленных.

- (c) Покажите, что полученные задачи линейного программирования дуальны друг другу.

«Сведите» здесь значит, что по **любому** решению задачи линейного программирования вы должны уметь восстановить решение исходной задачи за размер ответа ($\mathcal{O}(E)$ для потока, $\mathcal{O}(V)$ для разреза).

Дополнительные задачи

5. Правила игры «Volunteer's dilemma» на n игроков следующие: стратегия каждого игрока — вероятность, с которой он вызывается добровольцем. Если в игре не вызвалось ни одного добровольца, все получают 0, иначе добровольцы получают по $(1 - c)$, а все остальные — по 1. $0 < c < 1$.
- (a) Найдите симметричное равновесие Нэша.
 - (b) Найдите какое-то нетривиальное несимметричное равновесие Нэша для $n > 2$ («нетривиальное» значит не одна единица и остальные — нули).
 - (c) Найдите все равновесия Нэша (и докажите, что других нет).

12 Поиск подстроки в строке

12.1 Практика

1. Найдите число различных подстрок строки. $\mathcal{O}(n^2)$.
2. Найдите все периоды строки. $\mathcal{O}(n)$.
3. Найдите позицию строки в ее суффиксном массиве. $\mathcal{O}(n)$.
4. Научиться искать за $\mathcal{O}(n+m)$ образец в строке, если между образцом и найденной подстрокой допустимо различие:
 - (a) в один символ.
 - (b) в два символа.
5. Дана строка длины n и образец длины m над алфавитом $[0 \dots k-1]$. Найдите образец в строке за $\mathcal{O}(n+m+k)$, если допустимо в образце применять к алфавиту:
 - (a) циклический сдвиг алфавита.
 - (b) произвольную перестановку алфавита
6. Дан набор непустых строк суммарной длины n над алфавитом Σ .
 - (a) Отсортируйте за время $\mathcal{O}(n \log |\Sigma|)$.
 - (b) Покажите, что за $\mathcal{O}(n)$ нельзя.
 - (c) Отсортируйте за $\mathcal{O}(n + |\Sigma|)$, если Σ — небольшие целые числа.
7. Дан массив a длины n из m -битных чисел. Найдите пару a_i, a_j : $a_i \oplus a_j = \max$. $\mathcal{O}(nm)$.
8. Дан словарь слов суммарной длины L и текст T . Длины слов в словаре не более l . Представьте текст в виде конкатенации минимального количества словарных слов за время $\mathcal{O}(L + l|T|)$. Слова можно использовать более одного раза.
9. Даны словарь и текст. Нужно уметь обновлять ответ **online** при добавлении символов в конец текста.
 - (a) Пересчитайте суммарное число вхождений слов из словаря в текст за $\mathcal{O}(1)$.
 - (b) Поддерживайте множество всех вхождений слов из словаря в текст. Пересчёт за время $\mathcal{O}(1 + |\Delta A|)$, где ΔA — приращение ответа после добавления очередного символа.
10. Дан словарь слов суммарной длины L . За время $\mathcal{O}(L)$ определите, существует ли бесконечная строка, не содержащая ни одно словарное слово как подстроку.
11. Дан словарь, постройте структуру, чтобы быстро отвечать на запросы:
 - (a) $\text{get}(s)$ — число строк в словаре, которые начинаются с s .
 - (b) $\text{get}(s)$ — число строк в словаре, которые начинаются с s , заканчиваются на $\overleftarrow{s}$.
 - (c) $\text{get}(s, t)$ — число строк в словаре, которые начинаются с s , заканчиваются на t , $|s| = |t|$.
 - (d) $\text{get}(s, t)$ — число строк в словаре, которые начинаются с s , заканчиваются на t .

Дополнительные задачи

12. За $\mathcal{O}(n)$ построить строку с данной:
 - (a) Z-функцией.
 - (b) префикс-функцией.
13. Придумайте, как про строку s детерминировано проверить за $\mathcal{O}(|s|)$, что:
 - (a) она лексикографически меньше любого ее суффикса.
 - (b) она лексикографически не больше любого ее циклического сдвига.

12.2 Домашнее задание

1. Для каждого префикса строки найдите количество его префиксов, равных его суффиксу. $\mathcal{O}(n)$.
2. Постройте строку с данной префикс-функцией (или скажите, что это невозможно). И докажите, что ваш алгоритм правильно работает.
 - (a) За $\mathcal{O}(n^2)$.
 - (b) За $\mathcal{O}(n)$.
3. Преобразовать Z-функцию в префикс-функцию без промежуточного восстановления строки за $\mathcal{O}(n)$.
4. Дана строка s , нужно за $\mathcal{O}(|s|)$ найти такую минимальную строку t , что st – палиндром.
5. Дан набор n непустых строк s_i . Для каждой s_i найдите $\min$ по длине префикс, который не является префиксом других строк. $\mathcal{O}(\sum |s_i|)$.

Дополнительные задачи

6. За одну секунду в конец изначально пустого текста дописывается случайная буква (равномерное распределение). Чему равно матожидание времени T , когда первый раз s станет подстрокой выписанного текста?
Сведите к линейной системе на частичный балл и к замкнутой формуле вычисляемой за $\mathcal{O}(|s|)$ арифметических операций — на полный.

13 Суффиксные структуры

13.1 Практика

1. Построить за линейное время:
 - (a) суффиксный массив по суффиксному дереву
 - (b) попарные lcp по суффиксному массиву
 - (c) суффиксное дерево по суффиксному массиву
2. Научитесь online лексикографически сравнивать подстроки строки за $\mathcal{O}(1)$, имея суффиксный массив строки и $\mathcal{O}(n)$ времени на предобработку.
3. Постройте строку, у которой суффиксный массив совпадает с данным. $\mathcal{O}(n)$.
4. Найдите самую длинную общую подстроку k строк суммарной длины n за $\mathcal{O}(n)$.
 - (a) Суффиксным массивом.
 - (b) Суффиксным деревом.
5. Посчитайте количество различных подстрок строки.
6. Найдите самую длинную подстроку, входящую в заданную строку дважды за $\mathcal{O}(n)$.
 - (a) Вхождения могут пересекаться.
 - (b) Вхождения не могут пересекаться.
7. У вас были строка $s_0 s_1 \dots s_{n-1}$ длины n и ее суффиксный массив — массив позиций начал суффиксов, отсортированных в лексикографическом порядке. Символы строки — натуральные числа, не превосходящие n . Под покровом темноты подлые враги прокрались и стерли из массива все числа, делящиеся на 3, так, что даже дырок не осталось! Придумайте алгоритм, восстанавливающий суффиксный массив за время $\mathcal{O}(n)$.

Дополнительные задачи

8. Дан словарь суммарной длины n , постройте за $\mathcal{O}(n \log n)$ структуру, чтобы быстро отвечать на запросы
 - (a) $\text{get}(s)$ — число строк в словаре, которые начинаются с s , заканчиваются на $\text{rev}(s)$. $\mathcal{O}(|s|)$.
 - (b) $\text{get}(s, t)$ — число строк в словаре, которые начинаются с s , заканчиваются на t , $|s| = |t|$. $\mathcal{O}(|s|)$.
 - (c) $\text{get}(s, t)$ — число строк в словаре, которые начинаются с s , заканчиваются на t . $\mathcal{O}(|s| + |t| + \log n)$.
9. Постройте строку **над минимальным алфавитом**, у которой суффиксный массив совпадает с данным. $\mathcal{O}(n)$.
10. Дан набор строк s_i . Для каждой s_i найдите min по длине подстроку, которая не встречается в других. $\mathcal{O}(n)$.
11. Дан последний столбец отсортированного массива циклических сдвигов строки. Также дан номер исходной строки в этом порядке. Восстановить строку.
 - (a) $\mathcal{O}(n^2)$
 - (b) $\mathcal{O}(n)$

13.2 Домашнее задание

1. Для строки длины n найдите усреднённое по всем (не только последовательным) парам суффиксов значение lcp с помощью суффиксного массива. $\mathcal{O}(n)$.
2. Дана строка s длины n . Найдите её самую длинную подстроку, разворот которой также входит в s как подстрока, с помощью суффиксного массива. $\mathcal{O}(n)$.
3. Дан набор строк s_i суммарной длины n . Для каждой s_i найдите минимальную по длине **подстроку**, которая не встречается в других. $\mathcal{O}(n)$.
Решите двумя способами: суффиксными деревом и массивом.
4. Даны k непустых строк суммарной длины n . Найдите за $\mathcal{O}(n)$ среди их общих подстрок p -ю лексикографически. k — **не** константа.
 - (a) Алфавит константного размера.
 - (b) Алфавит — небольшие целые числа.

Дополнительные задачи

5. Дана строка s длины n . Придумайте структуру данных со следующими операциями:
 - $\text{init}(s)$ — построить структуру данных. $\mathcal{O}(n \log n)$.
 - $\text{get}(p, l, r)$ — перечислить вхождения p как подстроки s в позициях от l -ой до r -ой. $\mathcal{O}(|p| + \log n + m)$ (здесь m — размер ответа).
6. Даны словарь непустых строк суммарной длины L , строка s длины n и число p . Пусть X — множество всех строк длины как минимум p , являющихся подстрокой какого-то словарного слова. Найдите в s все позиции, которые не покрыты ни одним вхождением строки из X . $\mathcal{O}(n + L)$.